STAT6206 Tutorial

Department of Statistics
CUHK

February 11, 2025

Overview

1. Motivating example
2. Estimating with “R2WinBUGS”
3. Two real examples
4. MH algorithm

Section 1

Motivating example

Poisson GLM Example

A hospital chain investigates factors influencing monthly infection counts(y) across 100 branches.
Suppose the true model to generate y is:

$$y \sim \text{Poisson}(\mu),$$
$$\log(\mu) = \beta_0 + \beta_1 x_1 + \beta_2 x_2,$$

- x_1: Standardized hospital size (beds)
- x_2: Handwashing compliance rate
- $\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2) = (0.5, 1, -1)^T$

Key Questions:

- How to recover true β parameters from simulated data?

Section 2

Estimating with “R2WinBUGS”

What is WinBUGS

- To introduce R2WinBUGS, we first introduce WinBUGS.
- WinBUGS (Windows version of Bayesian inference Using Gibbs Sampler) is primarily developed using MCMC techniques such as the Gibbs sampler and the Metropolis-Hastings (MH) algorithm.
- It has been shown that under broad conditions, this software can provide reliable Bayesian statistics such as the estimates of parameters and latent variables in the proposed model.

Download WinBUGS

- The WinBUGS can be downloaded from the website: `https://www.mrc-bsu.cam.ac.uk/software/bugs-project`
- The WinBUGS manual that provides brief introductions is also available online.
- When you finish installing WinBUGS, remember to download the key file for unrestricted use. `https://drive.google.com/file/d/11n4gF6H6IPjGXYfwy7VlIGUW5cd9KOdR/view`

Install WinBUGS

- Exit all other programs currently running.
- Copy WinBUGS14.exe to your computer.
- Go into Explorer and double-click on WinBUGS14.exe.
- Follow the instructions in the dialog box.
- You should have a new directory called WinBUGS14 within Program Files.
- Inside the WinBUGS14 directory is a program called WinBUGS14.exe.
- Right-click on the WinBUGS icon to create a shortcut, select ‘create shortcut’, then drag this shortcut to the desktop.
- Double-click on WinBUGS14.exe to run WinBUGS14.

Note

- There appears to be a problem with installing WinBUGS and/or various patches in Windows 7 and 10.
- Windows 7 and 10 do not seem to like anyone overwriting files in the "C:\Program Files" directory (regardless of permissions). Hence, we recommend that WinBUGS should be installed elsewhere, e.g. "D:\WinBUGS".
- For more details, see WinBUGS_AppendixB.pdf.

R2WinBUGS Package

- WinBUGS is an interactive program, and it is not convenient to use it to do simulation directly.
- However, WinBUGS can be run in batch mode using scripts, and the R package R2WinBUGS uses this feature and provides tools to call WinBUGS directly after manipulation in R.
- Furthermore, it is possible to work on the results after importing them back into R.
- The implementation of R2WinBUGS is mainly based on the R function 'bugs()', which takes data and initial values as input.
- It automatically writes a WinBUGS script, calls the model, and saves the simulation for easy access in R.

Poisson GLM Example

A hospital chain investigates factors influencing monthly infection counts(y) across 100 branches.
Suppose the true model to generate y is:

$$y \sim \text{Poisson}(\mu),$$
$$\log(\mu) = \beta_0 + \beta_1 x_1 + \beta_2 x_2,$$

- x_1: Standardized hospital size (beds)
- x_2: Handwashing compliance rate
- $\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2) = (0.5, 1, -1)^T$

Key Questions:

- How to recover true β parameters from simulated data?

Poisson GLM Example - Generate data

- Ref: glm_count.R
- We first set the true values.
- To ensure accuracy, we typically generate no less than 100 data sets based on the true model. However, for presentation purposes, we are generating only 1 datasets in this case.

```
# Load required packages
library(R2WinBUGS)

set.seed(123)  # For reproducibility
n <- 300       # Sample size
true_beta <- c(0.5, 1, -1)  # True parameter values

# Generate covariates
x1 <- rnorm(n)
x2 <- rnorm(n)
```

Poisson GLM Example - Generate data

- We generate data with rpois().

```
# Calculate linear predictor and generate outcomes
linear_predictor <- true_beta[1] + true_beta[2]*x1 + true_beta
    [3]*x2
mu <- exp(linear_predictor)
y <- rpois(n, lambda = mu)
```

Poisson GLM Example - Write WinBUGS model Code

- Then, we need to specify the model code in WinBUGS.

```
# WinBUGS model definition
poisson_model <- function() {
  for(i in 1:N) {  # Loop over each observation
    y[i] ~ dpois(mu[i])  # Specify that y[i] follows a Poisson
        distribution with mean mu[i]
    log(mu[i]) <- beta[1] + beta[2]*x1[i] + beta[3]*x2[i]  #
       Log-linear relationship for mu
  }
  for(j in 1:3) {  # Loop over the number of beta coefficients
    beta[j] ~ dnorm(0, 0.001)  # Vague normal priors for beta
       coefficients
  }
}
```

How to write WinBUGS code

- **Stochastic Nodes**: In the text-based model description, stochastic nodes are denoted by the node name, followed by a tilde symbol (~), the distribution name, and a comma-separated list of parent nodes enclosed in brackets. For example:

$$y[i,j] \sim \text{dnorm}(\mu[i,j], \psi[j])$$

  This indicates that $y[i,j]$ is a stochastic node that follows a normal distribution, where the mean is $\mu[i,j]$ and the variance is $\frac{1}{\psi[j]}$.

- **Logical Nodes**: Logical nodes are represented by the node name followed by a left arrow (<-) and a logical expression involving its parent nodes. For example:

$$\mu[i,l] \leftarrow u[l] + \eta[i,l]$$

How to write WinBUGS codes

- **Arrays and Indexing**: Arrays are indexed using terms within square brackets. The four basic operators $+, -, *, /$ can be used, along with appropriate bracketing, to calculate an integer function as an index. For example: $y[i, j], y[i + l, j + l], y[i * j, k]$.
- **Repeated Structures**: Repeated structures are defined using a "for-loop." The syntax for this is:
  for (i in a:b) { list of statements }

Poisson GLM Example - Load data and initial value

- Then, we loaded generated data into WinBUGS.
- Set the initial value for MCMC chains.

```
# Prepare data for WinBUGS
bugs_data <- list(
    N = n,
    y = y,
    x1 = x1,
    x2 = x2
)
# Set initial values
inits <- function() {
  list(
    beta = rnorm(3, mean = 0, sd = 0.5)  # Random initial
       values)
}
```

Poisson GLM Example - Run WinBUGS

- Then, we specify the parameters to estimate. Here, we only focus on beta.

```
# Parameters to monitor
parameters <- c("beta")
```

- Finally, we use the bugs() function to run the code.
- A WinBUGS window will open, and R will become unresponsive during this time. The model will run in WinBUGS, which may take a while. You will see activity in the Log window within WinBUGS. Once the process is complete, the WinBUGS window will close, and R will resume functioning normally.

Poisson GLM Example - Run WinBUGS

```r
# Run WinBUGS
results <- bugs(
  data = bugs_data,                    # Model data
  inits = inits,                       # Initial parameter values
  parameters.to.save = parameters,  # Parameters to save
  model.file = poisson_model,       # WinBUGS model file
  n.chains = 1,                        # Number of MCMC chains
  n.iter = 10000,                      # Total iterations
  n.burnin = 5000,                     # Burn-in period
  n.thin = 2,                          # Thinning interval
  debug = FALSE,                       # Debug mode
  bugs.directory = "D:/WinBUGS14/",  # WinBUGS installation
     path
  working.directory = getwd()       # Current working directory
)
```

Poisson GLM Example - Analyze Result

- Summarize the estimation results based on the replications(Here we set REPS $= 1$, usually $100+$). Analyze results using:

$$\text{Bias} = \frac{1}{REPS} \sum_{i=1}^{REPS} (\hat{\theta}_i - \theta_{\text{true}})$$

$$\text{RMSE} = \sqrt{\frac{1}{REPS} \sum_{i=1}^{REPS} (\hat{\theta}_i - \theta_{\text{true}})^2}$$

- If bias and RMSE are close to 0, it indicates that the estimations are accurate and reliable.

Poisson GLM Example - Analyze Result

```r
# Storage matrix for results
beta_estimates <- matrix(NA, nrow = REP, ncol = 3)
colnames(beta_estimates) <- c("beta0", "beta1", "beta2")
beta_estimates[sim, ] <- results$summary[1:3, "mean"]

# Calculate performance metrics
bias <- colMeans(beta_estimates, na.rm = TRUE) - true_beta
empirical_se <- apply(beta_estimates, 2, sd, na.rm = TRUE)
rmse <- sqrt(colMeans((beta_estimates - matrix(true_beta, REP,
    3, byrow = TRUE))^2, na.rm = TRUE))
```

Poisson GLM Example - Result analysis

```
> print(results_table, row.names = FALSE, digits = 4)
 Parameter True_Value Mean_Estimate     Bias      SE    RMSE
     beta0        0.5         0.444 -0.05596 0.06011 0.07760
     beta1        1.0         1.015  0.01464 0.03019 0.03071
     beta2       -1.0        -1.013 -0.01303 0.04513 0.04242
```

- All the bias are close to 0, indicating excellent unbiasedness of the Bayesian estimator.
- All the RMSE are close to 0, suggesting estimation errors are primarily driven by variance rather than systematic bias.

More Details

- R2WinBUGS package manual
  `https://cran.r-project.org/web/packages/R2WinBUGS/index.html`
- Sturtz, S., Ligges, U., & Gelman, A. (2005). R2WinBUGS: a package for running WinBUGS from R. Journal of Statistical software, 12, 1-16. `https://cran.r-project.org/web/packages/R2WinBUGS/vignettes/R2WinBUGS.pdf`

Section 3

Two real examples

Example 1: Headache data

This data set is obtained from a two-treatment, multiple-period crossover trial in which the number of headaches per week was repeatedly measured for 27 patients. In the first period, each patient received the placebo. In the remaining periods, the patients received a placebo (P) or aspartame (A) in random order, using the double-blind crossover treatment design.(see headache.R)
Let

- y_{it} — # of headaches for patient i in period t
- d_{it} — # of days for patient i in period t
- x_{it} - treatment received by patient i in period t.
- 0 = placebo, 1 = aspartame

Example 1: Headache data

Two models are considered:
M_1: Poisson GLMM

$$\log \frac{\mu_{it}}{d_{it}} = \beta_0 + \beta_1 x_{it} + u_i, \quad \text{or} \quad \mu_{it} = d_{it} \exp\left(\beta_0 + \beta_1 x_{it} + u_i\right)$$

where $y_{it} \mid u_i \sim \text{Poisson}\left(\mu_{it}\right)$, and $u_i \sim N\left(0, \sigma^2\right)$.
M_2 : Poisson GLM

$$\log \frac{\mu_{it}}{d_{it}} = \beta_0 + \beta_1 x_{it}, \text{ or } \mu_{it} = d_{it} \exp\left(\beta_0 + \beta_1 x_{it}\right)$$

Example 1: Headache data - Load data

```r
library(mvtnorm)      # For multivariate normal distributions
library(R2WinBUGS)    # Interface between R and WinBUGS

# Data Preparation
N <- 27  # Number of subjects/groups

# Read headache data with column names
headache <- read.table("headache.txt", col.names = c("d", "y",
    "x", "NT"))
attach(headache)

# Reshape data into matrices for WinBUGS compatibility
x <- matrix(x, byrow = TRUE, ncol = 5)  # covariate x
d <- matrix(d, byrow = TRUE, ncol = 5)  # dose variable d
y <- matrix(y, byrow = TRUE, ncol = 5)  # outcome y
```

Example 1: Headache data - Preperation

```r
# Model 1: Poisson GLMM (Mixed Effects Model)
parameters <- c("beta", "sgm")  # Parameters to monitor

# Initial values for two chains
init1 <- list(beta = rep(-1, 2), psi = 1)
init2 <- list(beta = rep(1, 2), psi = 2)
inits <- list(init1, init2)

# Data list for WinBUGS
data <- list(N = N, NT = NT, x = x, d = d, y = y)
```

Example 1: Headache data - Write model GLMM

```
# Define GLMM model
glmmp_model <- function() {
  # Loop over each individual
  for(i in 1:N) {
    # Loop over each time point for the individual
    for(t in 1:NT[i]) {
      # Calculate the mean (mu) for the Poisson distribution
      mu[i,t] <- exp(beta[1] + beta[2] * x[i,t] + u[i]) * d[i,
         t]
      # Define the outcome variable y as following a Poisson
         distribution
      y[i,t] ~ dpois(mu[i,t])
    } # end of time point loop
```

Example 1: Headache data - Write model GLMM

```
    # Define the random effect u for each individual
    u[i] ~ dnorm(0, psi)
  } # end of individual loop

  # Prior distribution for the precision parameter psi
  psi ~ dgamma(9, 4)
  # Convert precision to standard deviation
  sgm <- 1 / psi

  # Prior distributions for the regression coefficients (beta)
  for(j in 1:2) {
    beta[j] ~ dnorm(0, 0.01)
  } # end of beta loop
}
```

Example 1: Headache data - Use WinBUGS ans save result for GLMM

```
# Run GLMM model
model1<-bugs(data,inits,parameters, model.file=glmmp_model,
             n.chains=2,n.iter=8000,n.burnin=3000,n.thin=1,
             bugs.directory="D:/WinBUGS14/",
             working.directory=getwd(),
             debug=FALSE)

# Extract results for GLMM
Ebeta1 <- model1$mean$beta # Posterior means of beta
SEbeta1 <- model1$sd$beta # Posterior standard deviations
Esgm1 <- model1$mean$sgm  # Posterior mean of random effects
   SD
SEsgm1 <- model1$sd$sgm   # Posterior SD of random effects SD
```

Example 1: Headache data - Write model GLM

```
glm_model <- function() {
  for(i in 1:N) {   # Loop over individuals
    # Loop over time points for each individual
    for(t in 1:NT[i]) {
      # Calculate mean for Poisson distribution
      mu[i,t] <- exp(beta[1] + beta[2] * x[i,t]) * d[i,t]
      # Define outcome variable y
      y[i,t] ~ dpois(mu[i,t])
    } # end of time loop
  } # end of individual loop
  # Prior distributions for regression coefficients
  for(j in 1:2) {
    beta[j] ~ dnorm(0, 10)  # Normal prior
  } # end of beta loop
}
```

Example 1: Headache data - Use WinBUGS ans save result for GLM

```
# Run GLM model
model2 <- bugs(data, initg, parametersg,
               model.file = glm_model,
               n.chains = 2,
               n.iter = 8000,
               n.burnin = 3000,
               n.thin = 1,
               bugs.directory = "D:/WinBUGS14/",
               working.directory = getwd(),
               debug = FALSE)

# Extract results for GLM
Ebeta3 <- model2$mean$beta      # Posterior means of beta
SEbeta3 <- model2$sd$beta       # Posterior standard deviations
```

Example 1: Headache data

- DIC values: The DIC is a model selection criterion where lower values indicate a better fit. The Poisson GLM has a DIC of 471, while the Poisson GLMM has a significantly lower DIC of 392. This suggests that the Poisson GLMM model provides a better fit to the data compared to the Poisson GLM.
- pD: The posterior pD value is 20.76 for the GLMM and 1.77 for the GLM. A higher pD indicates a more complex model with better predictive power. The GLMM model, therefore, demonstrates a more nuanced understanding of the data.

Model	Parameter	Estimate	SE
GLMM	beta1	-1.7224136	0.16936798
GLMM	beta2	0.2782956	0.14276333
GLM	beta1	-1.3861047	0.08199174
GLM	beta2	0.1272343	0.12532611
GLMM	sgm	0.5021683	0.13843410

Model	DIC	pD	Beta1_Est	Beta1_SE	Beta2_Est	Beta2_SE
Poisson GLMM	392	20.76	-1.72	0.169	0.278	0.143
Poisson GLM	471	1.77	-1.39	0.082	0.127	0.125

Example 2: Child's Wheeze Data

This example is based on a well-known data set from the Six Cities study, a longitudinal study of the health effects of air pollution. 537 children from Ohio, at ages 7, 8, 9, and 10 years were investigated (Chib and Greenberg, 1998). Variables include:

- wheezing status ($1 = yes, 0 = no$) $\rightarrow Y_{it}$.
- mother's baseline smoking status ($1 =$ yes, $0 = no$) $\rightarrow S$
- children's age $\rightarrow a_{it}$.

Example 2: Child's Wheeze Data

Let y_{it} denote the wheeze status ($1 = yes, 0 = no$) for child i at age t, and $\eta_{it} = E(y_{it} \mid \mathbf{u}_i) = p_{it}$. A GLMM with the following linear predictor is considered: where

$$\eta_{it} = \beta_0 + \beta_1 s_i + \beta_2 a_{it} + u_i + v_{it}$$

- s_i - mother's smoking habit at baseline
- a_{it} - child i at age t
- u_i and v_{it} - independent random effects with distributions $N\left[0, \sigma_u^2\right]$ and $N\left[0, \sigma_{vt}^2\right]$

Example 2: Child's Wheeze Data

Using the logit link and the linear predictor η_{it}, the GLMM can be rewritten as : where

$$\text{logit}\left(p_{it}\right) = \mathbf{x}_{it}^T\boldsymbol{\beta} + \mathbf{z}_{it}^T\mathbf{u}_i,$$

- $\mathbf{x}_{it} = \left(1, s_i, a_{it}\right)^T, \boldsymbol{\beta} = \left(\beta_0, \beta_1, \beta_2\right)^T,$
- $\mathbf{u}_i = \left(u_i, v_{i1}, v_{i2}, v_{i3}, v_{i4}\right)^T,$
- $\mathbf{z}_{i1} = \left(1, 1, 0, 0, 0\right)^T, \mathbf{z}_{i2} = \left(1, 0, 1, 0, 0\right)^T,$
- $\mathbf{z}_{i3} = \left(1, 0, 0, 1, 0\right)^T, \mathbf{z}_{i4} = \left(1, 0, 0, 0, 1\right)^T,$
- $\mathbf{u}_i \sim N(\mathbf{0}, \boldsymbol{\Sigma})$, with $\boldsymbol{\Sigma} = \text{diag}\left(\sigma_u^2, \sigma_{v1}^2, \sigma_{v2}^2, \sigma_{v3}^2, \sigma_{v4}^2\right)$

Example 2: Child's Wheeze Data - Preperation

```r
library(mvtnorm)      # For multivariate normal distributions
library(R2WinBUGS)    # Bayesian inference using WinBUGS
library(geepack)      # Contains Ohio respiratory dataset

# Load and restructure Ohio childhood respiratory data
data(ohio)
N <- 537     # Number of children
NT <- 4      # Number of time points

# Create matrices for longitudinal analysis:
age <- matrix(ohio$age, byrow = TRUE, ncol = NT)  # Age matrix
smoke <- ohio$smoke[(0:536)*4 + 1]                # Maternal
   smoking status (baseline)
y <- matrix(ohio$resp, byrow = TRUE, ncol = NT)   #
   Respiratory status matrix
```

Example 2: Child's Wheeze Data - Write model

```
glm_model <- function() {
  for(i in 1:N) {
    for(t in 1:NT) {
      eta[i,t] <- beta[1] + beta[2] * s[i] + beta[3] * a[i,t]
         + u[i] + v[i,t] # linear predictor (eta)
      # Calculate probability (p) using the logistic function
      p[i,t] <- exp(eta[i,t]) / (1 + exp(eta[i,t]))
      y[i,t] ~ dbern(p[i,t]) # outcome variable y
      # Define random effect v for each time point
      v[i,t] ~ dnorm(0, psi_v[t])
    } # end of time loop
    # Define random effect u for each individual
    u[i] ~ dnorm(0, psi_u)
  } # end of individual loop
}
```

Example 2: Child's Wheeze Data - Write model

```
  # Prior distribution for the precision of random effect u
  psi_u ~ dgamma(9, 4)
  sgm_u <- 1 / psi_u  # Standard deviation for u
  # Priors for regression coefficients
  for(j in 1:3) {
    beta[j] ~ dnorm(0, 0.01)  # Normal prior with mean 0 and
       precision 0.01
  } # end of beta loop
  # Loop over time points for precision of random effect v
  for(k in 1:NT) {
    psi_v[k] ~ dgamma(9, 4)  # Prior for precision of v
    sgm_v[k] <- 1 / psi_v[k]  # Standard deviation for v
  }
}
```

Example 2: Child's Wheeze Data - Load data

```
# Model Specification
parameters <- c("beta", "sgm_u", "sgm_v")  # Parameters to
   monitor

# Initial Values for 2 Chains
init1 <- list(beta = rep(-1,3), psi_u = 1, psi_v = rep(1,NT))
init2 <- list(beta = rep(1,3), psi_u = 2, psi_v = rep(2,NT))
inits <- list(init1, init2)

# WinBUGS Data Input
data <- list(N = N, NT = NT, a = age, s = smoke, y = y)
```

Example 2: Child's Wheeze Data — Run and save

```
# Run Bayesian Model
model <- bugs(data, inits, parameters, model.file = glm_model,
              n.chains = 2,          # Number of MCMC chains
              n.iter = 3000,         # Total iterations
              n.burnin = 1000,       # Burn-in period
              n.thin = 1,            # Thinning interval
              bugs.directory = "D:/WinBUGS14/",
              working.directory = getwd(), debug = FALSE)
# Extract Results
Ebeta <- model$mean$beta       # Fixed effects mean
SEbeta <- model$sd$beta        # Posterior SDs of fixed effects
Esgm_u <- model$mean$sgm_u     # Between-child SD
SEsgm_u <- model$sd$sgm_u
Esgm_v <- model$mean$sgm_v     # Between-time SD
SEsgm_v <- model$sd$sgm_v
```

Example 2: Child's Wheeze Data - Results and Analysis

Please refer to Child_Wheeze.R for further details.

- **Baseline Risk:** The intercept ($\beta_0 = -3.14$) indicates a baseline probability of $\exp(-3.14)/[1 + \exp(-3.14)] = 4.2\%$ for respiratory issues in newborns of non-smoking mothers (95% CI: 3.0-5.5%).
- **Maternal Smoking:** A positive association ($\beta_1 = 0.40 \pm 0.27$) was observed, indicating increased risk for infants of smoking mothers compared to non-smokers.
- **Age Effect:** Each additional year of age is associated with a reduced risk ($\beta_2 = -0.18 \pm 0.07$).

```
        Parameter Estimate    SE
1       Intercept   -3.138 0.211
2 Maternal Smoking    0.395 0.267
3             Age   -0.178 0.071
```

Section 4

MH algorithm

Outline

- Motivation: Sampling from non-standard distribution
- Metropolis-Hastings Algorithm
  - Key idea
  - Implementation
- Example

Basic Sampling Techniques

- Inverse method
- Rejection method
- Importance sampling
- Gibbs sampling

Motivating example

Let $Y = \{y_{it}; t = 1, \ldots, T_i, i = 1, \ldots, n\}$ and $U = (u_1, \ldots, u_n)$. In model (3.19), assume $g(\cdot)$ is a canonical link, that is

$$\theta_{it} = g(\mu_{it}) = x_{it}^T \boldsymbol{\beta} + u_i.$$

Assign prior distributions for $\boldsymbol{\beta}$ and $\boldsymbol{\Sigma}$ as follows:

$$\boldsymbol{\beta} \sim N(\boldsymbol{\beta}_0, \boldsymbol{\Sigma}_0), \quad \boldsymbol{\Sigma} \sim IW_q[R_0, r_0],$$

where $IW_q[\cdot, \cdot]$ denotes a q-dimensional inverted Wishart distribution, and q is the dimension of u_i.

Full Conditional Distributions

The main task of Bayesian inference is to work on $p(\boldsymbol{\beta}, \boldsymbol{\Sigma}, U|Y)$. However, $p(\boldsymbol{\beta}, \boldsymbol{\Sigma}, U|Y)$ is intractable. Thus, the Gibbs sampler is used to iteratively sample from their conditional posterior (i.e., full conditional) distributions through:

- $p(\boldsymbol{\beta}|Y, U, \boldsymbol{\Sigma})$,
- $p(\boldsymbol{\Sigma}|Y, U, \boldsymbol{\beta})$,
- $p(U|Y, \boldsymbol{\beta}, \boldsymbol{\Sigma})$.

The full conditional distribution of $\boldsymbol{\beta}$:

$$p(\boldsymbol{\beta}|\cdot) \propto p(Y|U, \boldsymbol{\beta})p(\boldsymbol{\beta}) = \prod_{i=1}^{n}\prod_{t=1}^{T_i} \exp\left\{\frac{y_{it}\theta_{it} - b(\theta_{it})}{a(\phi)} + c(y_{it}; \phi)\right\} \tag{1}$$

$$\times \exp\left\{-\frac{1}{2}(\boldsymbol{\beta} - \boldsymbol{\beta}_0)^T \boldsymbol{\Sigma}_0^{-1}(\boldsymbol{\beta} - \boldsymbol{\beta}_0)\right\}. \tag{2}$$

Full Conditional Distributions

The full conditional distribution of $\mathbf{\Sigma}$:

$$p(\mathbf{\Sigma}|Y, U, \boldsymbol{\beta}) \sim IW_q[UU^T + R_0^{-1}, n + r_0].$$

The full conditional distribution of U:

$$p(U|Y, \boldsymbol{\beta}, \mathbf{\Sigma}) = \prod_{i=1}^{n} p(u_i|y_i, \boldsymbol{\beta}, \mathbf{\Sigma}) \propto \prod_{i=1}^{n} p(y_i|u_i, \boldsymbol{\beta}) p(u_i|\mathbf{\Sigma})$$

$$= \prod_{i=1}^{n} \left\{ \prod_{t=1}^{T_i} \exp\left\{ \frac{y_{it}\theta_{it} - b(\theta_{it})}{a(\phi)} + c(y_{it}; \phi) \right\} \times \exp\left\{ -\frac{1}{2} u_i^T \mathbf{\Sigma}^{-1} u_i \right\} \right\}.$$

$p(\boldsymbol{\beta}|Y, U, \mathbf{\Sigma})$ and $p(U|Y, \boldsymbol{\beta}, \mathbf{\Sigma})$ are not standard and complex! Thus, A hybrid algorithm combining the Gibbs sampler and the Metropolis-Hastings (MH) algorithm is used to implement the posterior sampling.

Metropolis-Hastings Algorithm

- **Objective**: Sample from a known (but nonstandard) density function.
- **Key Idea**: Sample a candidate from a standard distribution (proposal distribution) and accept the candidate with some probability (acceptance probability).

Metropolis-Hastings Algorithm

1. Give an initial value: x_0.
2. At the t-th MH iteration with a current value x_t, a new candidate x' is sampled from a proposal distribution $q(x'|x_t)$ which depends on x_t.
3. Calculate the acceptance probability:

$$\alpha = \min\left(1, \frac{p(x')q(x_t|x')}{p(x_t)q(x'|x_t)}\right).$$

4. Generate a uniform random number $u \sim U(0,1)$; if $u < \alpha$, accept the candidate x' (i.e., set $x_{t+1} = x'$); otherwise, set $x_{t+1} = x_t$.
5. Repeat Steps 2-4 until convergence.

Remarks

1. Theoretically, the proposal distribution can be arbitrary; however, it is wise to choose a standard distribution which (i) approximates to the target distribution; (ii) can be sampled from easily.
2. When the proposal distribution is symmetric with respect to x' and x_t, the acceptance probability may reduce to:

$$\alpha = \min\left(1, \frac{p(x')}{p(x_t)}\right).$$

3. The proposal distribution may include a tuning parameter that controls the acceptance rate.
4. The optimal acceptance rate is chosen on a problem-by-problem basis (Gelman et al. 1995).

Gelman, A., Roberts, G. O., and Gilks, W. R. Efficient Metropolis jumping rules. In J. M. Bernardo, J. O. Berger, A. P. Dawid and A. F. M. Smith (Eds), Bayesian Statistics 5, pp. 599-607. Oxford: Oxford University Press, 1995.

Example: GLMM

The target distribution is defined as:

$$p(\boldsymbol{\beta}|\cdot) \propto p(Y|U,\boldsymbol{\beta})p(\boldsymbol{\beta}) = \prod_{i=1}^{n}\prod_{t=1}^{T_i} \exp\left\{\frac{y_{it}\theta_{it} - b(\theta_{it})}{a(\phi)} + c(y_{it};\phi)\right\} \tag{3}$$

$$\times \exp\left\{-\frac{1}{2}(\boldsymbol{\beta}-\boldsymbol{\beta}_0)^T\boldsymbol{\Sigma}_0^{-1}(\boldsymbol{\beta}-\boldsymbol{\beta}_0)\right\}. \tag{4}$$

To facilitate sampling from the target distribution, we define a proposal distribution $q(\boldsymbol{\beta}'|\boldsymbol{\beta})$. A common choice for the proposal distribution is a multivariate normal distribution:

$$q(\boldsymbol{\beta}'|\boldsymbol{\beta}) \sim N(\boldsymbol{\beta}, \sigma_\beta^2\boldsymbol{\Sigma}_\beta).$$

Remarks:

1. $\boldsymbol{\Sigma}_\beta$ is the Hessian matrix of $-\log p(\boldsymbol{\beta}|\cdot)$ with respect to the $\boldsymbol{\beta} = 0$.
2. σ^2_β is the tuning parameter which is chosen such that the average acceptance rate is approximately 0.25 or more.

Acceptance Probability

The acceptance probability for transitioning from $\boldsymbol{\beta}$ to $\boldsymbol{\beta}'$ is given by:

$$\alpha = \min\left(1, \frac{p(\boldsymbol{\beta}'|Y)q(\boldsymbol{\beta}|\boldsymbol{\beta}')}{p(\boldsymbol{\beta}|Y)q(\boldsymbol{\beta}'|\boldsymbol{\beta})}\right).$$

This formulation ensures that we properly account for both the likelihood of the proposed and current states as well as the proposal distribution.

The End